L M U ☀ C M S I 3 8 0 2

LANGUAGES AND AUTOMATA II

HOMEWORK #3 Due: 2025-03-21

Learning Objectives

In this assignment you will demonstrate:

- ✅ The ability to write a semantic analyzer for a mid-sized, statically-typed language
- ✅ Knowledge of static analysis in programming language theory
- ✅ Basic familiarity with types, functions, and concurrency
- ✅ The ability to distinguish different types of language-processing errors (lexical, context-free, static contextual, runtime)
- ✅ Competency in growing and maintaining a large Node.js project
- ✅ The ability to deploy a website hosted on GitHub Pages

Readings, Videos, and Self-Study

Although your team will be turning in only one submission, every team member is individually responsible for each of the following:

- ✅ Do any of the assigned readings and watchings from the first two assignments that you did not get to.
- ✅ Work through the course notes on How to Write a Compiler (up to the section entitled *Optimization*).
- ✅ Read Chapters 4—6 of [Mogenson]
- ✅ Read this entire overview of Java Language changes from versions 9 through 23. The purpose of this reading is to get you familiar with modern programming languages features. There is a lot to this, so skim when it gets too detailed.
- ✅ Skim The Wikipedia page on Static Program Analysis
- ✅ Skim the OWASP brief article on Static Code Analysis (from a security perspective)
- ✅ Watch This video (you can play at 1.5x)
- ✅ Watch any videos embedded in the relevant course notes, as time allows

Submission Instructions

As usual, turn in a single submission for your entire group via BrightSpace. Remember to not simply partition the work among team members, as this reduces your learning.

Your team's BrightSpace submission should be an attached PDF or text file with:

- ✅ The names of every student
- ✅ For each student, an affidavit that each of the readings and watchings above were individually completed
- ✅ Answers to each of the exercises below, numbered and neatly typeset. For problems involving code to write, host your code on Replit or OneCompiler (or similar) and provide the link to the hosting site in your BrightSpace submission.
- ✅ A link to the public GitHub repository hosting your team's project

Exercises

1. Classify the following as a syntax error, static semantic (contextual) error, or not a compile time error. In the case where code is given, assume all identifiers are declared, have the expected type, and are in scope. All items refer to the Java language.

    a. `x+++-y`

    b. `x---+y`

    c. incrementing a read-only variable

    d. code in class C accessing a private field from class D

    e. Using an uninitialized variable

    f. Dereferencing a null reference

    g. `null instanceof C`

    h. `!!x`

    i. `x > y > z`

    j. `if (a instanceof Dog d) {...}`

    k. `var s = """This is weird""";`

    l. `switch = 200;`

    m. `x = switch (e) {case 1->5; default->8;};`

2. How do JavaScript and Rust treat the following:

```
let x = 3;
let x = 3;
```

3. Describe how the languages Java and Ruby differ in their interpretations of the meaning of the keyword `private`. You can use an AI chatbot for help, but please trim down the long-winded applications those tools are known for, and give a concise explanation that proves you truly understand the difference.

4. Some languages do not require the parameters to a function call to be evaluated in any particular order. Is it possible that different evaluation orders can lead to different arguments being passed? If so, give an example to illustrate this point, and if not, prove that no such event could occur.

5. Describe in your own words how the Carlos language allows handles recursive structs. Describe what kinds of restrictions the language definition imposes and why. Describe how the compiler enforces the restrictions. Write well. Use technical vocabulary accurately. An AI assistant can help you get your grammar and spelling right, though it is unlikely to get the right answer.

6. Some languages do not have loops. Write a function, using tail recursion (and no loops) to compute the minimum value of an array or list in Python, C, JavaScript, and in either Go, Erlang, or Rust (your choice). Obviously these languages probably already have a min-value-in-array function in a standard library, but the purpose of this exercise is for you to demonstrate your understanding of tail recursion. Your solution must be in the classic functional programming style, that is, it must be stateless. Use parameters, not nonlocal variables, to accumulate values. Assume the array or list contains floating-point values.

7. Your friend creates a little JavaScript function to implement a count down, like so:

```JS
function countDownFrom10() {
  let i = 10;
  function update() {
    document.getElementById("t").innerHTML = i;
    if (i-- > 0) setTimeout(update, 1000);
  }
  update();
}
```

Your other friend says “Yikes, you are updating a non-local variable! Here is a better way:”

```JS
function countDownFromTen() {
  function update(i) {
    document.getElementById("t").innerHTML = i;
    if (i-- > 0) setTimeout(update(i), 1000);
  }
```

```
  update(10);
}
```

What does your second friend's function do when called? Why does it fail? Your friend is on the right path though. Fix their code and explain why your fix works.

8. Find as many linter errors as you can in this Java source code file (C.java):



```
import java.util.HashMap;

class C {
    static final HashMap<String, Integer> m = new HashMap<String, Integer>();

    static int zero() {
        return 0;
    }

    public C() {
    }
}
```

You can use SonarLint or FindBugs or FindSecBugs or PMD or whatever you prefer. You might even need to use a combination of tools because it is possible no tool finds them all. (Please note you are not expected to already know what all the issues are here. The idea is to practice with tools and have good discussions with teammates. Find as many as you can, and read and understand each problem that is reported to you so you learn (1) what kinds of potential bugs and security problems can exist even in compilable and runnable code, and (2) the kinds of things that a static analyzer *can* detect.)

For Your Project

Continue your compiler project in the public GitHub repository you created in the first assignment. You will be expanding your repo to have the following:

```
.
├── .gitignore
├── README.md
├── LICENSE
├── package.json
├── .prettierrc.json
├── docs
|   └── ...                    -- now with the companion website
├── examples
|   └── ...                    -- add more example programs
├── src
|   ├── (yourlanguagename).js
```

```
|   ├── (yourlanguagename).ohm
|   ├── compiler.js
|   ├── core.js              -- new!
|   ├── parser.js            -- as before
|   ├── analyzer.js          -- with a completed static analyzer!
|   ├── optimizer.js         -- still just the stub function
|   └── generator.js         -- still just the stub function
└── test
    ├── compiler.test.js     -- just test that you can parse and analyze
    ├── parser.test.js       -- as before, but improve from HW2 feedback
    └── analyzer.test.js     -- new!
```

Your tasks for this assignment are twofold. First, you are to implement the static analyzer for your compiler. Second, you are to start a companion website for your language.

- Implement the static analyzer in your analyzer.js file. Since you're using Ohm, the analysis will be overlaid into the existing Ohm semantics object that you wrote to prepare the AST in the previous assignment. For this assignment, you'll be simply enhancing this object. Here are a few things to be thinking about for your analyzer:

  - Scope resolution
  - Proper contextual use of `return`, `break`, etc.
  - Type checking
  - Type inference
  - Parameter matching
  - Pattern exhaustiveness
  - Access controls

  Make sure your language has a non-trivial number of contextual rules to be enforced at compile time. Generally speaking, your language should feature static types. If it does not, make up for the lack of type checking with something really special, like really a really sophisticated concurrency mechanism that will require more sophisticated code generation and optimization in later assignments.

- Make sure your README file talks about the required static checking. (If your language is a "dynamic" language without static checks, at least list which checks are being deferred to run time.)

- Add tests to the analyzer.test.js file. Follow the style of testing from the Bella and Carlos compilers. You need checks that show that contextually correct programs generate the proper representation, and that all form of static errors are caught.

- For your language website (a.k.a. a "Home Page") for your programming language using GitHub Pages. The site's page (or pages) should consist of a single page and:

  - Be rather pretty, CSS-wise (you can use a template, of course).
  - Tell the story of your language in no more than three paragraphs.

    - Include examples of programs in your language. You need at least five complete, semantically correct, example programs, that cover every syntactic form of your language, and most if not all of the interesting semantic-level checks.
    - Include developer bios (and optionally pictures), because that's kind of fun.
    - Include a link to the actual GitHub repo of the compiler.
    - Also add a link to this site from the README of the GitHub repo.

Grading Rubric

To help you get a good score, here are all the things you will be graded on.

- Exercises (48 pts)
    - ✅ Exercise 1 (13 pts)
    - ✅ Exercise 2 (5 pts)
    - ✅ Exercise 3 (5 pts)
    - ✅ Exercise 4 (5 pts)
    - ✅ Exercise 5 (5 pts)
    - ✅ Exercise 6 (5 pts)
    - ✅ Exercise 7 (5 pts)
    - ✅ Exercise 8 (5 pts)
- Compiler Project (42 pts)
    - ✅ Your project can be cloned (1 pt)
    - ✅ In the project README, there is a link to your GitHub Pages site (1 pt)
    - ✅ I can run `npm test` immediately after cloning (1 pt)
    - ✅ All tests pass, and there are enough of them (3 pts)
    - ✅ The context is managed well—preferably in an object (7 pts)
    - ✅ The error handling is well-structured—nice check functions or methods (5 pts)
    - ✅ At least five interesting static checks (7 pts)
    - ✅ Analysis method are written like a pro—they work, and well named, etc. (6 pts)
    - ✅ You have at least 50 tests in your suite (5 pts, all or nothing)
    - ✅ The list of static errors are on the README or on your language website (3 pts)
    - ✅ Test coverage is 100% (3 pts, all or nothing)
- Project Website (10 pts)
    - ✅ The GH Pages site looks great: nice template, or hand-crafted CSS (3 pts)
    - ✅ Site has the story of your language in no more than three paragraphs (2 pts)
    - ✅ Site shows at least fives example programs your language (2 pts)
    - ✅ Site has developer names and short bios (2 pts)
    - ✅ Site points back to the repo (1 pt)